

Agenda

- Object-oriented languages
- Static/Dynamic dispatching
- vTables, iTables, monomorphisation

Object-oriented languages

- An object is:
 - A cohesive and heterogeneous collection of data and logic
 - A natural abstraction implementing an abstract data type

Abstract Data Type

- An opaque datatype
 - A Name
 - No implementation is revealed
 - Operations that create and eliminate values of the data type
- Examples
 - Modules (C, Pascal, OCaml, Haskell, etc)
 - Classes in OO languages (Simula, Smalltalk, C++, Java, C#, OCaml, etc)

```
// stack.h
#ifndef STACK_H
#define STACK_H

#include <stdbool.h>

typedef struct Stack *Stack;

Stack stack_create(void);
void stack_destroy(Stack s);

void stack_push(Stack s, int value);
int stack_pop(Stack s);
int stack_peek(const Stack s);

bool stack_is_empty(const Stack s);
int stack_size(const Stack s);

#endif
```


Object-oriented languages

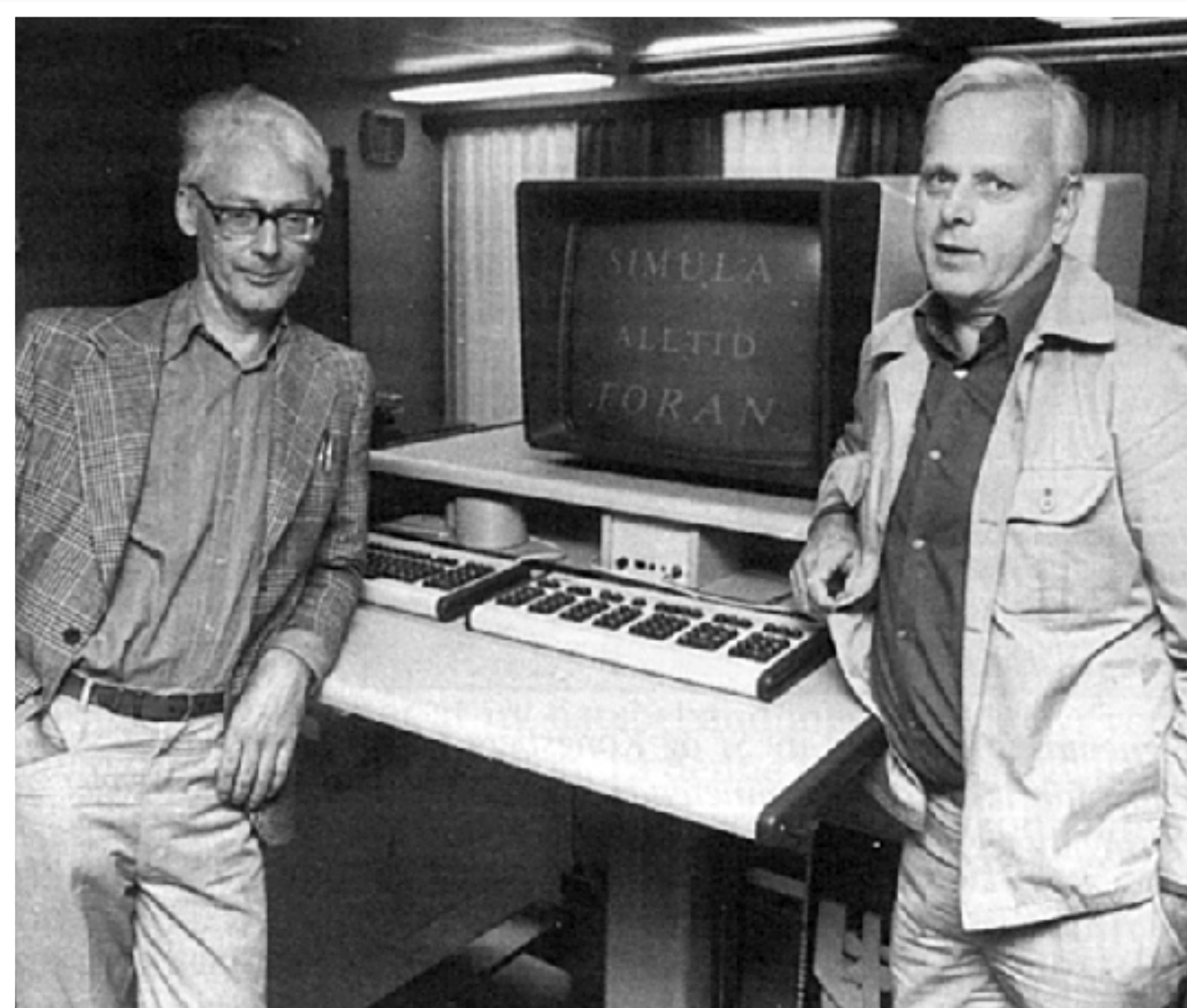
- An object is:
 - A cohesive and heterogeneous collection of data and logic
 - A natural abstraction implementing an abstract data type
 - A natural representation for Encapsulation and Information Hiding

Encapsulation and Information Hiding

- The internal representation is hidden from the usage context, allowing for reuse without the coupling to a particular representation. A first-class representation of modules (cf. C or OCaml).
- Objects: a recursive record containing data and logic. Equivalence is not structural; the **this/self** reference provides identity.
- Classes: a category of objects, an object factory, (sometimes) a type
- Interface: just a type, decoupled from implementations.
- Subtyping: Polymorphism by inclusion. Usually only in depth.
- Inheritance: reuse by inclusion of behaviour.

Simula 67 - Ole-Johan Dahl and Kristen Nygaard

- A superset of the Algol language
- First instance of classes, objects and inheritance
- Real-world modelling and simulations



<https://en.wikipedia.org/wiki/Simula>

```
Begin
  Class Glyph;
    Virtual: Procedure print Is Procedure print;;
  Begin
  End;

  Glyph Class Char (c);
    Character c;
  Begin
    Procedure print;
      OutChar(c);
    End;

  Glyph Class Line (elements);
    Ref (Glyph) Array elements;
  Begin
    Procedure print;
      Begin
        Integer i;
        For i:= 1 Step 1 Until UpperBound (elements, 1) Do
          elements (i).print;
        OutImage;
      End;
    End;

    Ref (Glyph) rg;
    Ref (Glyph) Array rgs (1 : 4);

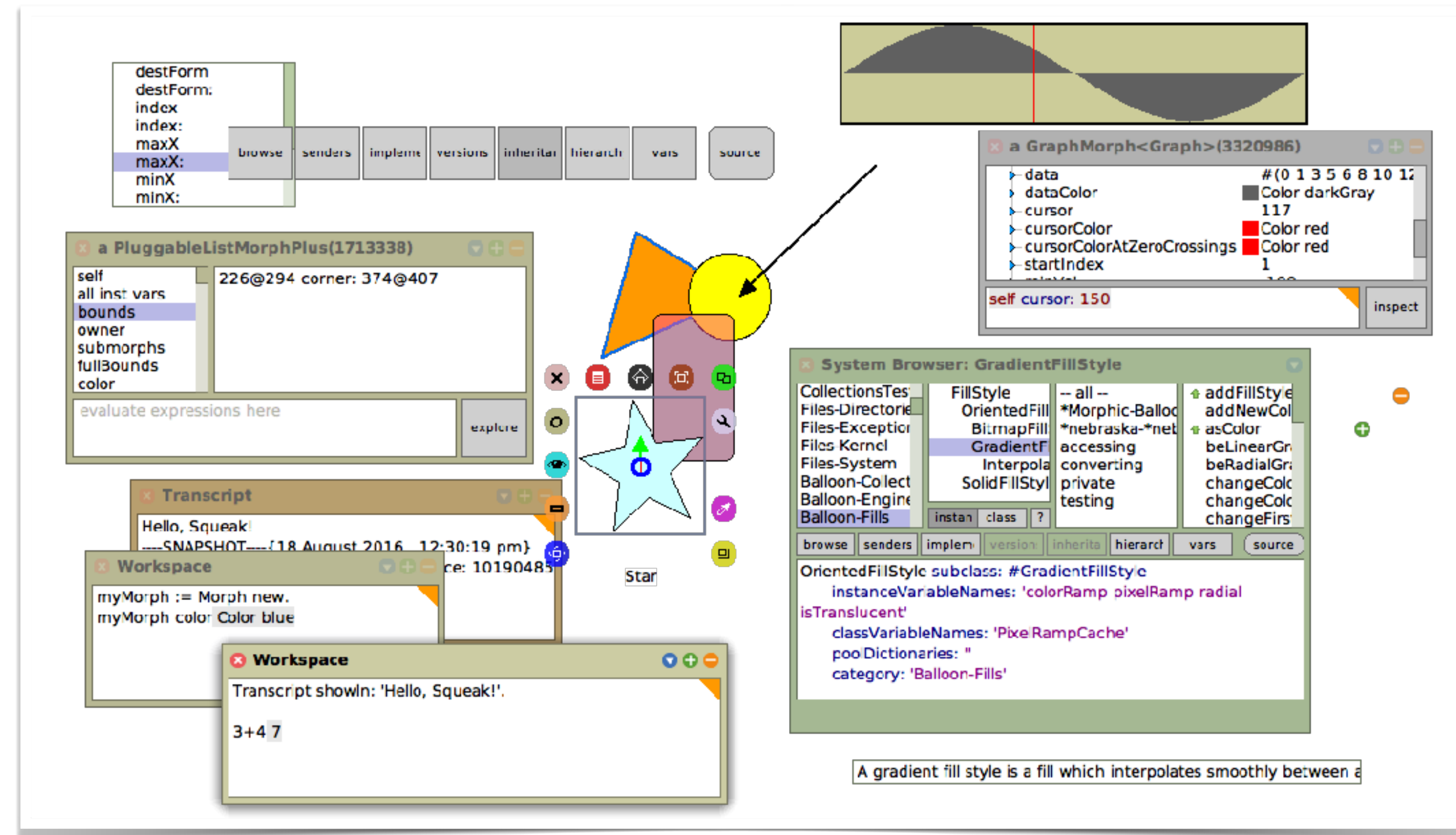
    ! Main program;
    rgs (1):- New Char ('A');
    rgs (2):- New Char ('b');
    rgs (3):- New Char ('b');
    rgs (4):- New Char ('a');
    rg:- New Line (rgs);
    rg.print;
  End;
```

Smalltalk & Squeak



Alan Kay, Dan Ingalls, Adele Goldberg, Ted Kaehler, Diana Merry, and Scott Wallace (1972, 1980)

- Pure object-oriented vision
 - Runs on a virtual machine
 - Everything is an object (even classes)
 - Definitions are stored in objects (no files) - Image-based persistence
 - Squeak is a live programming environment

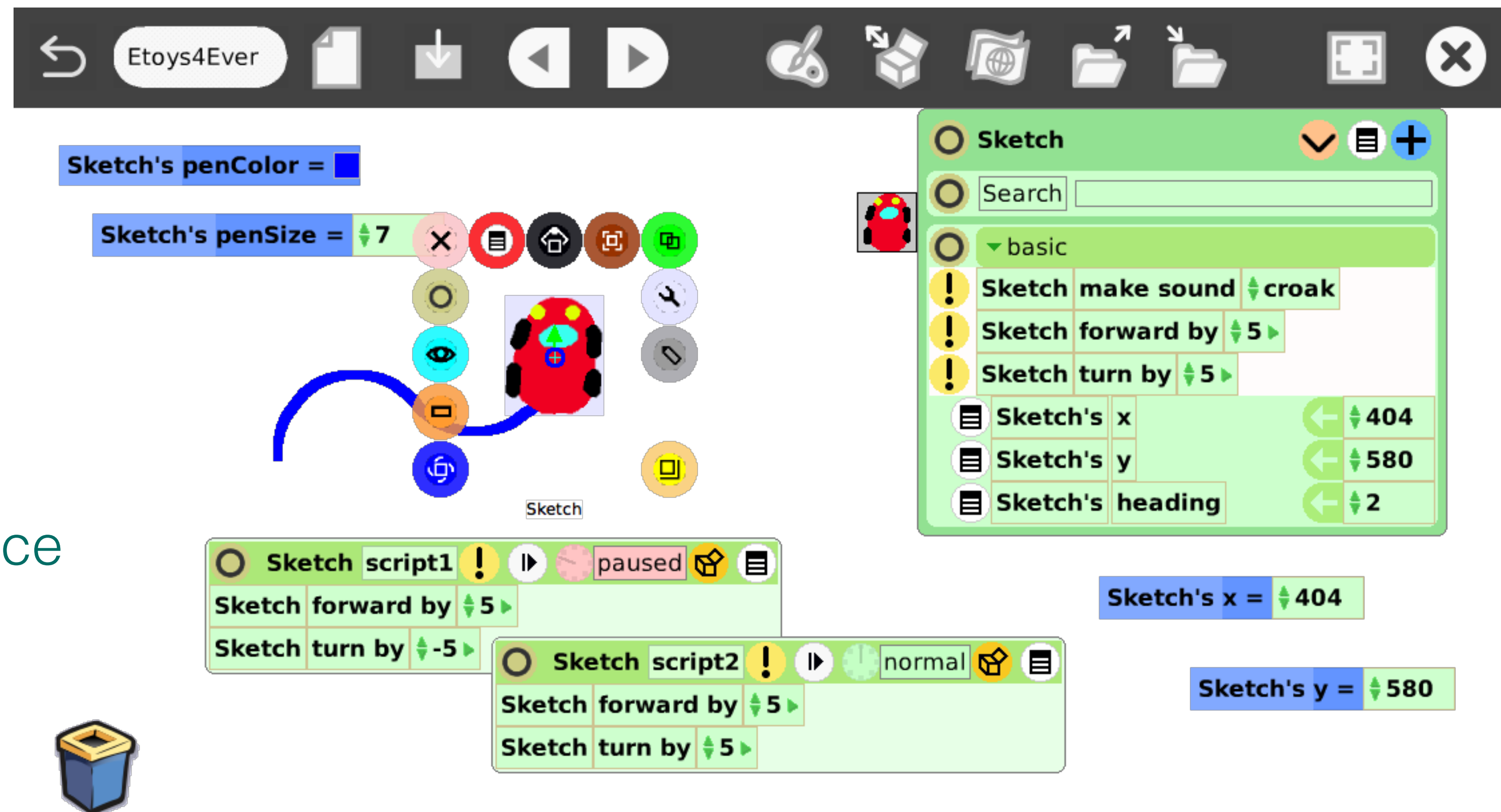


Smalltalk & Squeak



Alan Kay, Dan Ingalls, Adele Goldberg, Ted Kaehler, Diana Merry, and Scott Wallace (1972, 1980)

- Pure object-oriented vision
 - Runs on a virtual machine
 - Everything is an object (even classes)
 - Definitions are stored in objects (no files) - Image-based persistence
 - Squeak is a live programming environment



OO Encoding in OCaml

- Let's build the concept of an object from scratch

```
type counter = {count:int}  
let o = { count = 0 }
```

Comparing Object Encodings

Kim B. Bruce,¹ Luca Cardelli,² and Benjamin C. Pierce³

¹ Computer Science Department, Williams College
Williamstown, MA 01267, USA
kim@cs.williams.edu

² Digital SRC, 130 Lytton Ave
Palo Alto, CA 94301, USA
luca@pa.dec.com

³ Computer Science Department, Indiana University
Lindley Hall 215, Bloomington, IN 47405, USA
pierce@cs.indiana.edu

OO Encoding in OCaml

- Data and code in the same record, with recursion

Comparing Object Encodings

Kim B. Bruce,¹ Luca Cardelli,² and Benjamin C. Pierce³

¹ Computer Science Department, Williams College
Williamstown, MA 01267, USA
kim@cs.williams.edu

² Digital SRC, 130 Lytton Ave
Palo Alto, CA 94301, USA
luca@pa.dec.com

³ Computer Science Department, Indiana University
Lindley Hall 215, Bloomington, IN 47405, USA
pierce@cs.indiana.edu

```
type counter = {count:int; get: unit -> int}  
let o = let rec this = { count = 0; get = fun () -> this.count } in this
```

OO Encoding in OCaml

- Can we “modify the object”?
- Can immutable objects be returned... how?

Comparing Object Encodings

Kim B. Bruce,¹ Luca Cardelli,² and Benjamin C. Pierce³

¹ Computer Science Department, Williams College
Williamstown, MA 01267, USA
kim@cs.williams.edu

² Digital SRC, 130 Lytton Ave
Palo Alto, CA 94301, USA
luca@pa.dec.com

³ Computer Science Department, Indiana University
Lindley Hall 215, Bloomington, IN 47405, USA
pierce@cs.indiana.edu

```
type counter = {count:int; get: unit -> int; inc: unit -> counter}
let o = let rec this = { count = 0;
                        get = fun () -> this.count;
                        inc = fun () -> {count = this.count + 1; inc = this.}} in this
```


OO Encoding in OCaml

- Can we “modify the object”?
- Can immutable objects be returned by a builder function

```
type counter = {count:int;  
  get: unit -> int;  
  inc: unit -> counter;  
  dec: unit -> counter}  
  
let o =  
  let rec _init_Counter n =  
    let rec this =  
      { count = n;  
        get = (fun () -> this.count);  
        inc = (fun () -> _init_Counter (n+1));  
        dec = (fun () -> _init_Counter @@ n-1) }  
    in this  
  in _init_Counter 0
```

Comparing Object Encodings

Kim B. Bruce,¹ Luca Cardelli,² and Benjamin C. Pierce³

¹ Computer Science Department, Williams College
Williamstown, MA 01267, USA
kim@cs.williams.edu

² Digital SRC, 130 Lytton Ave
Palo Alto, CA 94301, USA
luca@pa.dec.com

³ Computer Science Department, Indiana University
Lindley Hall 215, Bloomington, IN 47405, USA
pierce@cs.indiana.edu

OO Encoding in OCaml

- Can we make mutable objects?

```
type counter = {count:int ref;  
                get: unit -> int;  
                inc: unit -> counter;  
                dec: unit -> counter}  
  
let o =  
  let rec _init_Counter n =  
    let rec this =  
      { count = ref(n);  
        get = (fun () -> !(this.count));  
        inc = (fun () -> this.count := !(this.count) + 1; this);  
        dec = (fun () -> this.count := !(this.count) - 1; this) }  
    in this  
  in _init_Counter 0
```

Comparing Object Encodings

Kim B. Bruce,¹ Luca Cardelli,² and Benjamin C. Pierce³

¹ Computer Science Department, Williams College
Williamstown, MA 01267, USA
kim@cs.williams.edu

² Digital SRC, 130 Lytton Ave
Palo Alto, CA 94301, USA
luca@pa.dec.com

³ Computer Science Department, Indiana University
Lindley Hall 215, Bloomington, IN 47405, USA
pierce@cs.indiana.edu

OO Encoding in OCaml

- Can we make mutable fields private?

```
type counter = {get: unit -> int;  
                inc: unit -> counter;  
                dec: unit -> counter}  
  
let o =  
  let _init_Counter n =  
    let count = ref n in  
    let rec this =  
      {  
        get = (fun () -> !count);  
        inc = (fun () -> count := !count + 1; this);  
        dec = (fun () -> count := !count - 1; this)  
      }  
    in this  
  in _init_Counter 0
```

Comparing Object Encodings

Kim B. Bruce,¹ Luca Cardelli,² and Benjamin C. Pierce³

¹ Computer Science Department, Williams College
Williamstown, MA 01267, USA
kim@cs.williams.edu

² Digital SRC, 130 Lytton Ave
Palo Alto, CA 94301, USA
luca@pa.dec.com

³ Computer Science Department, Indiana University
Lindley Hall 215, Bloomington, IN 47405, USA
pierce@cs.indiana.edu

OO Encoding in OCaml

- Can we make mutable fields private?

```
type counter = {  
  get: unit -> int;  
  inc: unit -> counter;  
  dec: unit -> counter  
}  
  
type stepper_counter = {  
  get: unit -> int;  
  inc: unit -> stepper_counter;  
  dec: unit -> stepper_counter;  
  set_step: int -> stepper_counter  
}
```

```
type counter_state = { count:int ref }  
  
let mk_counter state =  
  let count = state.count in  
  let rec this =  
    {  
      get = (fun () -> !count);  
      inc = (fun () -> count := !count + 1; this);  
      dec = (fun () -> count := !count - 1; this)  
    }  
  in this  
  
type stepper_counter_state = { count:int ref; step: int ref }  
let mk_stepper_counter {count;step} =  
  let super = mk_counter { count = count } in  
  let rec this =  
    {  
      get = super.get;  
      inc = (fun () -> count := !count + !step; this);  
      dec = (fun () -> count := !count - !step; this);  
      set_step = (fun s -> step := s; this)  
    }  
  in this
```


Encodings are a way to express the semantics of new language constructions (e.g. Simula, C++, Scala)

Dynamic dispatching

- Inclusion polymorphism is an essential part of OO languages.
- Dynamic dispatch is the capability to determine which method implementation to call at runtime (which object, which class).

Java

```
int printObject(Object o) {  
    System.out.println(o.toString());  
}
```

C

```
int f2() { return 0; }  
  
int f1() {  
    return f2();  
}
```

- Single and Multiple Dispatching: To determine at runtime which method to call depending on the types of the type of the parameters. Statically typed languages usually implement single (dynamic) dispatching, choose at compile time.

Virtual Method Tables (vTables)

```
#include <iostream>
using namespace std;

class A {
public:
    int m1() { return 0; }           // NOT virtual: resolved at compile time
    virtual int m2() { return m1(); } // virtual: slot in vtable; calls m1 statically
};

class B : public A {
public:
    int m1() { return 1; }           // hides A::m1, but does NOT override (no vtable slot)
    virtual int m3() { return m2(); } // calls m2() through vtable
};

int main() {
    B b;
    cout << b.m1() << "\n"; // 1 - static dispatch on B, picks B::m1
    cout << b.m3() << "\n"; // 0 - m3 -> m2 (vtable) -> m1 (static, A::m1); B::m1 is NOT in vtable

    // same story through a base pointer (no heap needed)
    A* p = &b;
    cout << p->m1() << "\n"; // 0 - static dispatch on A*, always A::m1
    cout << p->m2() << "\n"; // 0 - vtable dispatches to A::m2, which calls A::m1 statically
}
```

vTables encoding in C

```
typedef struct counter_t      counter_t;
typedef struct counter_vtable counter_vtable;
```

```
struct counter_vtable {
    int      (*get)(counter_t *self);
    counter_t (*inc)(counter_t *self);
    counter_t (*dec)(counter_t *self);
};
```

```
struct counter_t {
    counter_vtable *vtable;
    int            *count;
};
```

```
counter_t *make_counter(int initial) {
    counter_t *obj = malloc(sizeof(counter_t));
    int        *count = malloc(sizeof(int));
    *count      = initial;
    obj->vtable = &COUNTER_VTABLE;
    obj->count  = count;
    return obj;
}
```

```
static int counter_get(counter_t *self) {
    return *(self->count);
}
```

```
static counter_t *counter_inc(counter_t *self) {
    *(self->count) += 1;
    return self;
}
```

```
static counter_t *counter_dec(counter_t *self) {
    *(self->count) -= 1;
    return self;
}
```

```
static counter_vtable COUNTER_VTABLE = {
    .get = counter_get,
    .inc = counter_inc,
    .dec = counter_dec,
};
```


OO in Java

- The JVM supports objects and methods natively; the compiled code follows the structure. All methods are virtual by default.

```
public class Counter {  
    int counter;  
  
    public Counter() { this.counter = 0 ; }  
  
    public int get() { return this.counter ; }  
  
    public void inc() { this.counter++; }  
  
    public static void main() {  
        Counter c = new Counter();  
        c.inc();  
        System.out.println(c.get());  
    }  
}
```

```
public class Counter {  
    int counter;  
  
    public Counter();  
    Code:  
    0: aload_0  
    1: invokespecial #1  
    4: aload_0  
    5: iconst_0  
    6: putfield      #7  
    9: return  
  
    public int get();  
    Code:  
    0: aload_0  
    1: getfield      #7  
    4: ireturn  
  
    public void inc();  
    Code:  
    0: aload_0  
    1: dup  
    2: getfield      #7  
    5: iconst_1  
    6: iadd  
    7: putfield      #7  
   10: return  
  
    public static void main();  
    Code:  
    0: new           #8  
    3: dup  
    4: invokespecial #13  
    7: astore_0  
    8: aload_0  
    9: invokevirtual #14  
   12: getstatic     #17  
   15: aload_0  
   16: invokevirtual #23  
   19: invokevirtual #27  
   22: return  
}
```


OO in Java

- The object representation in memory starts with two words:
 - 8 bytes to encode GC age, Locking state, Hashcode.
 - 8 bytes: a pointer to the class object (contains the vtable)
- Method invocation in Java is performed by 5 different instructions
 - **invokevirtual**, methods in classes, uses a precompiled slot number on the vtable
 - **invokeinterface**, via interface types, includes a search in the table of names?
 - **invokespecial**, direct calls, for constructors and private methods
 - **invokestatic**, for static methods, no self object is given, no vtable.
 - **invokedynamic**, to invoke anonymous functions and support for dynamic languages.

InvokeInterface

- `Invokespecial` is a direct call to a function pointer.
- `Invokevirtual` requires a slot number in the vtable, 1 redirection.
- `Invokeinterface` requires a linear search in the iTable of the class, mapping interfaces to slots in classes' vtable.

```
public class Counter {
    int counter;

    public Counter();
    Code:
        0: aload_0
        1: invokespecial #1          // Method java/lang/Object."<init>":()V
        4: aload_0
        5: iconst_0
        6: putfield      #7          // Field counter:I
        9: return

    public int get();
    Code:
        0: aload_0
        1: getfield      #7          // Field counter:I
        4: ireturn

    public void inc();
    Code:
        0: aload_0
        1: dup
        2: getfield      #7          // Field counter:I
        5: iconst_1
        6: iadd
        7: putfield      #7          // Field counter:I
        10: return

    public static void main();
    Code:
        0: new           #8          // class Counter
        3: dup
        4: invokespecial #13         // Method "<init>":()V
        7: astore_0
        8: aload_0
        9: invokevirtual #14         // Method inc:()V
        12: getstatic    #17         // Field java/lang/System.out:Ljava/io/PrintStream;
        15: aload_0
        16: invokevirtual #23         // Method get:()I
        19: invokevirtual #27         // Method java/io/PrintStream.println:(I)V
        22: return
}
```

Monomorphization of call sites

- JIT compilers analyse the code, and optimize monomorphic calls, bypassing the vtable.
- Cache in the call site can also be done by mapping a class to a function pointer.